Q1.Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

```
//Question1

import java.io.*;
import java.util.*;

class Farmer {
    String name;
    String fruit;
    String city;
    int quantity;
}

public class FarmerData {
    public static void main(String[] args) throws Exception
{
        Scanner sc = new Scanner(System.in);

        Farmer f[] = new Farmer[10];

        // Input
        for(int i=0;i<10;i++) {
            f[i] = new Farmer();
            System.out.println("Enter details of Farmer " +
(i+1));

            System.out.print("Name: ");
            f[i].name = sc.next();

            System.out.print("Fruit: ");
            f[i].fruit = sc.next();
```

```
            System.out.print("City: ");
            f[i].city = sc.next();

            System.out.print("Quantity: ");
            f[i].quantity = sc.nextInt();
        }

        // Save before sorting
        FileWriter fw = new FileWriter("before.txt");
        fw.write("Name\tFruit\tCity\tQuantity\n");

        for(int i=0;i<10;i++) {
            fw.write(f[i].name + "\t" + f[i].fruit + "\t" +
f[i].city + "\t" + f[i].quantity + "\n");
        }
        fw.close();

        // Sorting (Bubble Sort)
        for(int i=0;i<9;i++) {
            for(int j=0;j<9-i;j++) {
                if(f[j].quantity > f[j+1].quantity) {
                    Farmer temp = f[j];
                    f[j] = f[j+1];
                    f[j+1] = temp;
                }
            }
        }

        // Save after sorting
        FileWriter fw2 = new FileWriter("after.txt");
        fw2.write("Name\tFruit\tCity\tQuantity\n");

        for(int i=0;i<10;i++) {
            fw2.write(f[i].name + "\t" + f[i].fruit + "\t" +
f[i].city + "\t" + f[i].quantity + "\n");
        }
        fw2.close();

        // Search
```

```
        System.out.print("\nEnter farmer name to search: ");
        String search = sc.next();

        boolean found = false;

        for(int i=0;i<10;i++) {
            if(f[i].name.equalsIgnoreCase(search)) {
                System.out.println("Found:");
                System.out.println(f[i].name + " " +
f[i].fruit + " " + f[i].city + " " + f[i].quantity);
                found = true;
            }
        }

        if(!found) {
            System.out.println("Farmer not found");
        }
    }
}
```

INPUT:-

Enter details of Farmer 1

Name: Raunak

Fruit: Mango

City: Pune

Quantity: 500

Enter details of Farmer 2

Name: Sanjay

Fruit: Apple

City: Delhi

Quantity: 300

Enter details of Farmer 3

Name: Aditya

Fruit: Banana

City: Mumbai

Quantity: 700

Enter details of Farmer 4

Name: Anish

Fruit: Orange

City: Nagpur

Quantity: 200

Enter details of Farmer 5

Name: Mohit

Fruit: Papaya

City: Nashik

Quantity: 900

Enter details of Farmer 6

Name: Ayush

Fruit: Grapes

City: Pune

Quantity: 400

Enter details of Farmer 7

Name: Pratik

Fruit: Mango

City: Jaipur

Quantity: 600

Enter details of Farmer 8

Name: Naruto

Fruit: Apple

City: Chandigarh

Quantity: 100

Enter details of Farmer 9

Name: Manas

Fruit: Banana

City: Bhopal

Quantity: 800

Enter details of Farmer 10

Name: Ratnesh

Fruit: Orange

City: Surat

Quantity: 350

OUTPUT:-

Enter farmer name to search: Ratnesh

Found:

Ratnesh Orange Surat 350

Q2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```java
//Question2

import java.util.Scanner;

class Fruit {
    String name;
    int quantity;
    String quality;

    Fruit(String name, int quantity, String quality) {
        this.name = name;

        // If quantity > 1000, store half
        if (quantity > 1000) {
            this.quantity = quantity / 2;
        } else {
            this.quantity = quantity;
        }

        this.quality = quality.toLowerCase();
    }

    void display() {
        System.out.println(name + " -> Quantity: " +
quantity + ", Quality: " + quality);
    }
}
```

```java
public class FruitStore {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input for fruits
        System.out.println("Enter Mango quantity and
quality:");
        int mq = sc.nextInt();
        String mql = sc.next();

        System.out.println("Enter Apple quantity and
quality:");
        int aq = sc.nextInt();
        String aql = sc.next();

        System.out.println("Enter Papaya quantity and
quality:");
        int pq = sc.nextInt();
        String pql = sc.next();

        // Create objects
        Fruit mango = new Fruit("Mango", mq, mql);
        Fruit apple = new Fruit("Apple", aq, aql);
        Fruit papaya = new Fruit("Papaya", pq, pql);

        // Display stored data
        System.out.println("\nStored Fruit Data:");
        mango.display();
        apple.display();
        papaya.display();

        // Search operation
        System.out.println("\nEnter fruit name to search:");
        String searchFruit = sc.next().toLowerCase();

        System.out.println("Enter quality to search:");
        String searchQuality = sc.next().toLowerCase();

        boolean found = false;
```

```
        // Check Mango
        if (searchFruit.equals("mango") &&
mango.quality.equals(searchQuality)) {
            mango.display();
            found = true;
        }

        // Check Apple
        if (searchFruit.equals("apple") &&
apple.quality.equals(searchQuality)) {
            apple.display();
            found = true;
        }

        // Check Papaya
        if (searchFruit.equals("papaya") &&
papaya.quality.equals(searchQuality)) {
            papaya.display();
            found = true;
        }

        if (!found) {
            System.out.println("Fruit with given quality not
found.");
        }

        sc.close();
    }
}
```

INPUT :-

Enter Mango quantity and quality:

1200 good

Enter Apple quantity and quality:

800 bad

Enter Papaya quantity and quality:

1500 average

Enter fruit name to search:

mango

Enter quality to search:

good

OUTPUT :-

Mango -> Quantity: 600, Quality: good

Q3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

```
//Question3

import java.util.Scanner;

class Orchard {
    String name;
    String fruit;
    String quality;
}

public class OrchardTree {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Orchard o[] = new Orchard[4];

        // Assign data
        o[0] = new Orchard(); o[0].name="A";
o[0].fruit="Apple"; o[0].quality="Good";
        o[1] = new Orchard(); o[1].name="B";
o[1].fruit="Litchi"; o[1].quality="Good";
        o[2] = new Orchard(); o[2].name="C";
o[2].fruit="Mango"; o[2].quality="Good";
        o[3] = new Orchard(); o[3].name="D";
o[3].fruit="Apple"; o[3].quality="Bad";

        // Tree View
        System.out.println("Tree View of Orchards:");
        System.out.println("Root");
        for(int i=0; i<4; i++) {
            System.out.println(" |-- Orchard " + o[i].name +
```

```
                                " (" + o[i].fruit + ", " +
o[i].quality + ")");
        }

        // Search
        System.out.print("\nEnter fruit to search: ");
        String f = sc.next();

        System.out.println("\nOrchards having " + f + ":");
        for(int i=0; i<4; i++) {
            if(o[i].fruit.equalsIgnoreCase(f)) {
                System.out.println("Orchard " + o[i].name);
            }
        }
    }
}
```

INTPUT :-

Enter fruit to search: Apple

OUTPUT :-

Tree View of Orchards:

Root

|-- Orchard A (Apple, Good)

|-- Orchard B (Litchi, Good)

|-- Orchard C (Mango, Good)

|-- Orchard D (Apple, Bad)

Orchards having Apple:

Orchard A

Orchard D

Q4. Take a map provided as a graph & display all the subtrees of connected points.

```
//Question4

public class GraphSubtree {
    static int n = 5;

    static void dfs(int graph[][], boolean visited[], int
node) {
        visited[node] = true;
        System.out.print(node + " ");

        for(int i=0;i<n;i++){
            if(graph[node][i]==1 && !visited[i]){
                dfs(graph, visited, i);
            }
        }
    }

    public static void main(String[] args) {

        int graph[][] = {
            {0,1,0,0,0},
            {1,0,0,0,0},
            {0,0,0,1,0},
            {0,0,1,0,0},
            {0,0,0,0,0}
        };

        boolean visited[] = new boolean[n];

        System.out.println("Subtrees:");

        for(int i=0;i<n;i++){
            if(!visited[i]){
                dfs(graph, visited, i);
                System.out.println();
```

```
            }
        }
    }
}
```

OUTPUT:-

Subtrees:

0 1

2 3

4

Q5. Enter entry point & goal point & produce the matching subtree of the graph.

```
//Question5

import java.util.*;

public class PathSearch {

    static boolean dfs(int graph[][], boolean visited[], int
node, int goal) {
        visited[node] = true;
        System.out.print(node + " ");

        if(node == goal)
            return true;

        for(int i=0;i<5;i++){
            if(graph[node][i]==1 && !visited[i]){
                if(dfs(graph, visited, i, goal))
                    return true;
            }
        }
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int graph[][] = {
            {0,1,0,0,0},
            {1,0,1,0,0},
            {0,1,0,1,0},
            {0,0,1,0,1},
            {0,0,0,1,0}
        };

        System.out.print("Enter start: ");
        int start = sc.nextInt();
```

```java
        System.out.print("Enter goal: ");
        int goal = sc.nextInt();

        boolean visited[] = new boolean[5];

        System.out.print("Path: ");
        if(!dfs(graph, visited, start, goal)){
            System.out.println("\nNo path found");
        }
    }
}
```

INPUT :-

Enter start: 0

Enter goal: 4

OUTPUT :-

Path: 0 1 2 3 4

Q6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
//Question6

public class ShortestPath {

    public static void main(String[] args) {

        int n = 5;

        int graph[][] = {
            {0,2,0,1,0},
            {2,0,3,0,0},
            {0,3,0,0,1},
            {1,0,0,0,2},
            {0,0,1,2,0}
        };

        int dist[] = new int[n];
        boolean visited[] = new boolean[n];

        // Initialize
        for(int i=0;i<n;i++){
            dist[i] = 999;
        }

        dist[0] = 0;

        for(int i=0;i<n-1;i++){
            int min = 999, u = 0;

            for(int j=0;j<n;j++){
                if(!visited[j] && dist[j] < min){
                    min = dist[j];
                    u = j;
                }
            }
```

```
            visited[u] = true;

            for(int v=0;v<n;v++){
                if(graph[u][v]!=0 && !visited[v]){
                    if(dist[u] + graph[u][v] < dist[v]){
                        dist[v] = dist[u] + graph[u][v];
                    }
                }
            }
        }

        System.out.println("Shortest distances from node
0:");
        for(int i=0;i<n;i++){
            System.out.println("0 -> " + i + " = " +
dist[i]);
        }
    }
}
```

OUTPUT :-

Shortest distances from node 0:

0 -> 0 = 0

0 -> 1 = 2

0 -> 2 = 5

0 -> 3 = 1

0 -> 4 = 3

Q7. Write a program to demonstrate the use of all keywords made for exception handling.

```
//Question7

import java.util.Scanner;

// Custom Exception
class MyException extends Exception {
    MyException(String msg) {
        super(msg);
    }
}

public class ExceptionDemo {

    // using throws
    static void checkNumber(int num) throws MyException {
        if(num < 0) {
            // using throw
            throw new MyException("Number cannot be
negative");
        } else {
            System.out.println("Valid number: " + num);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter a number: ");
            int n = sc.nextInt();

            checkNumber(n);

            // Arithmetic Exception
            int result = 10 / n;
```

```
            System.out.println("Result: " + result);

        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
        catch (MyException e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Finally block executed");
        }
    }
}
```

INPUT :-

Enter a number: 5

OUTPUT :-

Valid number: 5

Result: 2

Finally block executed

Q8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```
//Question8

import java.util.Scanner;

// Custom Exception
class NoMoneyException extends Exception {
    NoMoneyException(String msg) {
        super(msg);
    }
}

class Bank {
    double balance = 5000;

    void withdraw(double amount) throws NoMoneyException {
        if(amount > balance) {
            throw new NoMoneyException("Insufficient
Balance!");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawal Successful");
            System.out.println("Remaining Balance: " +
balance);
        }
    }
}

public class BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Bank b = new Bank();

        try {
            System.out.print("Enter amount to withdraw: ");
```

```
            double amt = sc.nextDouble();

            b.withdraw(amt);
        }
        catch (NoMoneyException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

INPUT :-

Enter amount to withdraw: 2000

OUTPUT :-

Withdrawal Successful

Remaining Balance: 3000.0

Q9. Write a program which enables to use a platform by a train one at a time.

```
//Question9

class Platform {

    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is entering the
platform");

        try {
            Thread.sleep(2000); // simulate time
        } catch (Exception e) {}

        System.out.println(trainName + " is leaving the
platform\n");
    }
}

class Train extends Thread {
    Platform p;
    String name;

    Train(Platform p, String name) {
        this.p = p;
        this.name = name;
    }

    public void run() {
        p.usePlatform(name);
    }
}

public class TrainDemo {
    public static void main(String[] args) {

        Platform p = new Platform();
```

```
        Train t1 = new Train(p, "Train 1");
        Train t2 = new Train(p, "Train 2");
        Train t3 = new Train(p, "Train 3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT :-

Train 1 is entering the platform

Train 1 is leaving the platform

Train 2 is entering the platform

Train 2 is leaving the platform

Train 3 is entering the platform

Train 3 is leaving the platform

Q10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

```
//Question10

class Restaurant {

    synchronized void placeOrder(String orderName) {
        System.out.println("Waiter takes order: " +
orderName);

        try {
            System.out.println("Chef is preparing: " +
orderName);
            Thread.sleep(2000); // simulate cooking time
        } catch (Exception e) {}

        System.out.println("Chef delivers: " + orderName +
"\n");
    }
}

class Waiter extends Thread {
    Restaurant r;
    String order;

    Waiter(Restaurant r, String order) {
        this.r = r;
        this.order = order;
    }

    public void run() {
        r.placeOrder(order);
    }
}

public class RestaurantDemo {
```

```java
    public static void main(String[] args) {

        Restaurant r = new Restaurant();

        Waiter w1 = new Waiter(r, "Pizza");
        Waiter w2 = new Waiter(r, "Burger");
        Waiter w3 = new Waiter(r, "Pasta");

        w1.start();
        w2.start();
        w3.start();
    }
}
```

OUTPUT :-

Waiter takes order: Pizza

Chef is preparing: Pizza

Chef delivers: Pizza

Waiter takes order: Burger

Chef is preparing: Burger

Chef delivers: Burger

Waiter takes order: Pasta

Chef is preparing: Pasta

Chef delivers: Pasta